ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ, ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΑΠΤΥΞΗ ΕΞΥΠΝΩΝ ΔΙΕΠΑΦΩΝ ΚΑΙ ΠΑΙΧΝΙΔΙΩΝ

ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ - PROJECT

Sonic the hedgehog:
The CSD Chronicles

ΔΙΔΑΣΚΩΝ: ΑΝΤΩΝΙΟΣ ΣΑΒΒΙΔΗΣ

Χειμερινό εξάμηνο 2024 - 2025

Περιεχόμενα

Contents

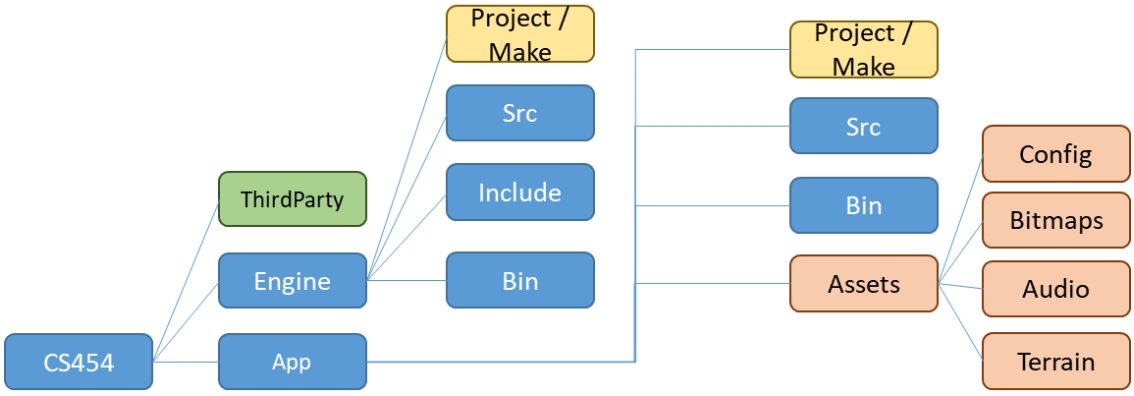
Εισαγωγή 3
Step1: Edit, Store, Load, Construct and Scroll a Tile Map 3
Step 2: Edit, Store, Load, Construct and Use a Grid Map 4
Step 2 Alt: Automatic Grid Computation 10
Step 3: Implement Layers and Parallax Scrolling 12
Step 4 : Spin Loops (προαιρετικό, default→bypass loop) 15
Action: Power-ups and enemies 20
Placing Powerups, Items and Bonuses 20
Placing NPCs 20
Supporting Pause / Resume 21
Action: Characters 22
Sonic 22
Enemies 23
Power-ups 25
Interactive Blocks - Entities 25
Action: Rings, damage and invincibility frames after damage. 26
Optional Configuration Parameters 27
Optional Extras 27
Ομαδική εργασία -παράδοση 28

Εισαγωγή

Η εργασία υλοποιείται σε βήματα, τα πρώτα είναι προπαρασκευαστικά και σκοπό έχουν να εξασφαλίσουν την έγκαιρη και σωστή υλοποίηση της βασικής μηχανής (game engine). Δεν εξετάζονται.

Κυρίαρχο χαρακτηριστικό της υλοποίησης είναι αφενός η διαφοροποίηση της μηχανής ως ξεχωριστό library, αφετέρου η υλοποίηση των πρώτων φάσεων ως ανεξάρτητων βημάτων (εάν θέλετε διακριτά projects και executable, αλλά δεν είναι απαραίτητο) τα οποία υλοποιούν και ελέγχουν βασικά χαρακτηριστικά της μηχανής, και επίσης μπορεί να υλοποιούν μερικές λειτουργίες που θα τις χρειαστείτε στην κατασκευή του κυρίως παιχνιδιού.

Η συνιστώμενη δομή του κώδικά σας (folder and project structure), πρέπει να είναι ως εξής.



Step1: Edit, Store, Load, Construct and Scroll a Tile Map

Τα βήματα που πρέπει να κάνετε είναι τα εξής:

1. Να βρείτε και να ***εγκαταστήσετε κάποιον free tile map editor*** ο οποίος μπορεί και αποθηκεύει τα maps σε κάποιο απλό text format. Ένας προτεινόμενος editor είναι ο Tiled, Version 1.11 (Stable)
    - https://www.mapeditor.org/
2. Να ***βρείτε κάποια έτοιμα maps και tile sets*** για αυτόν τον editor. Στην επιλογή σας σταθμίστε και τη διαθεσιμότητα τέτοιων maps. Ειδικά για την περίπτωση του Tiled, στο folder *UnitTest1/media* Project θα βρείτε ένα έτοιμο map καθώς και το αντίστοιχο tile set (πηγή https://beast-pixels.itch.io/overworld-tileset-grass-biome). Προσοχή, αυτό έχει δύο maps (layers). Μόνο το βασικό θα φορτώσετε (θα καταλάβετε ποιο είναι).
    - **Ωστόσο, είναι υποχρεωτικό να βρείτε και ένα ακόμη map και tile set**.
    - *Το μέγεθος του map τουλάχιστον σε μία από τις δύο διαστάσεις θα πρέπει να είναι 3-4 φορές το view window για να έχουμε επαρκές περιθώριο για το scrolling*.
    - Ο λόγος που διατίθενται τα έτοιμα maps είναι απλώς για να εξοικειωθείτε, καθώς στο τέλος πρέπει να κατασκευάσετε το map του sonic

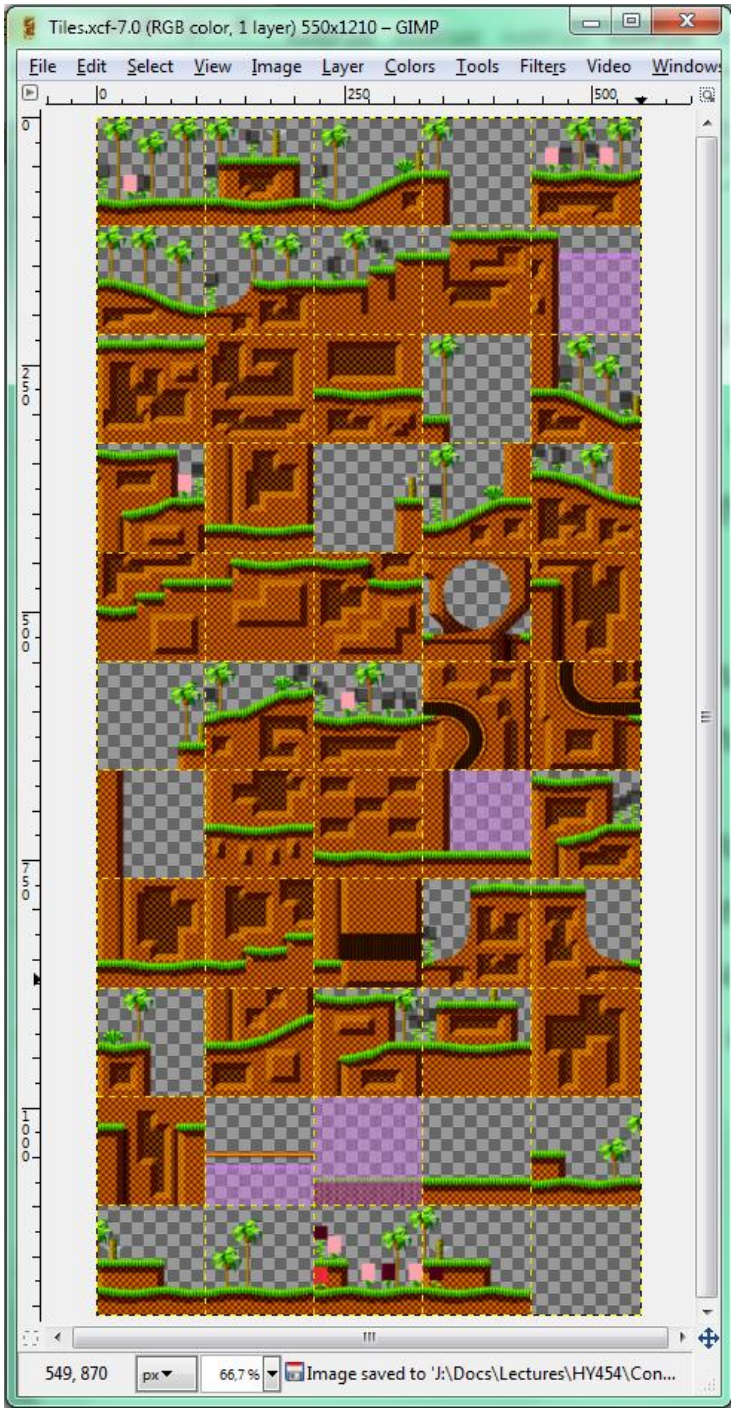
3. ***Να κάνετε export όλο το map σε text***. Ειδικά για τον Tiled, κάνετε export σε csv (Comma Separated Values), που επιτρέπει very fast and easy parsing. Μέσα στο csv format του Tiled εμφανίζονται indices από το tile set. Το Tiled αποδίδει ως indices συνεχόμενες αριθμούς που είναι η θέση ενός tile στο tile set, *αριστερά→δεξιά*, *πάνω→κάτω*.
4. ***Υλοποιήστε ένα custom parsing function*** για το text format του map editor (υλοποίηση της `ReadTextMap(…)`, και χρησιμοποιήστε το ώστε στο να κάνετε load το map.
5. ***Υλοποιήστε όλα τα σχετικά με το map*** (display function, scrolling, κλπ), πλην του grid.
6. ***Κάνετε initialize graphics σε window mode, resolution 640x480***, και ορίστε ως display area όλο το window και ως map view window το ίδιο επίσης.
7. Υλοποιήστε ένα **App** και **Game** class (namespace **app**) όπως σας έχει δοθεί από τις διαφάνειες (συνοδευτικός κώδικας).
8. ***Υλοποιήστε*** input logic χρησιμοποιώντας πληκτρολόγιο ή και mouse για να κάνει ***interactive scrolling*** το map view window με ένα offset - από το πληκτρολόγιο το <dx,dy> scroll offset θα είναι fixed, από το mouse το *<dx,dy>* του mouse pointer. Και στις δύο περιπτώσεις θα γίνεται applied και ένα multiplication factor { 0.5f, 1.0f, 1.5f , 2.0f }, με default value 1.0f
    - Για το mouse να απαιτείται clicked left button για να είναι ενεργοποιημένο το scrolling.
    - Με τα + και – πλήκτρα κάνετε next / previous το scroll multiplication factor με looped boundaries δηλαδή *previous(0.5)=2.0* και *next(2.0)=0.5* – προσοχή: μόνο όταν έχουμε release button, δηλαδή εδώ δε ελέγχεται το αν είναι pressed καθώς έτσι θα αλλάζει πολύ γρήγορα. Με το zero key θα επανέρχεται στο default value 0.5
    - Από το πληκτρολόγιο τα home / end button θα κάνουν set to view window στο top-left / bottom-right position αντίστοιχα.

Step 2: Edit, Store, Load, Construct and Use a Grid Map

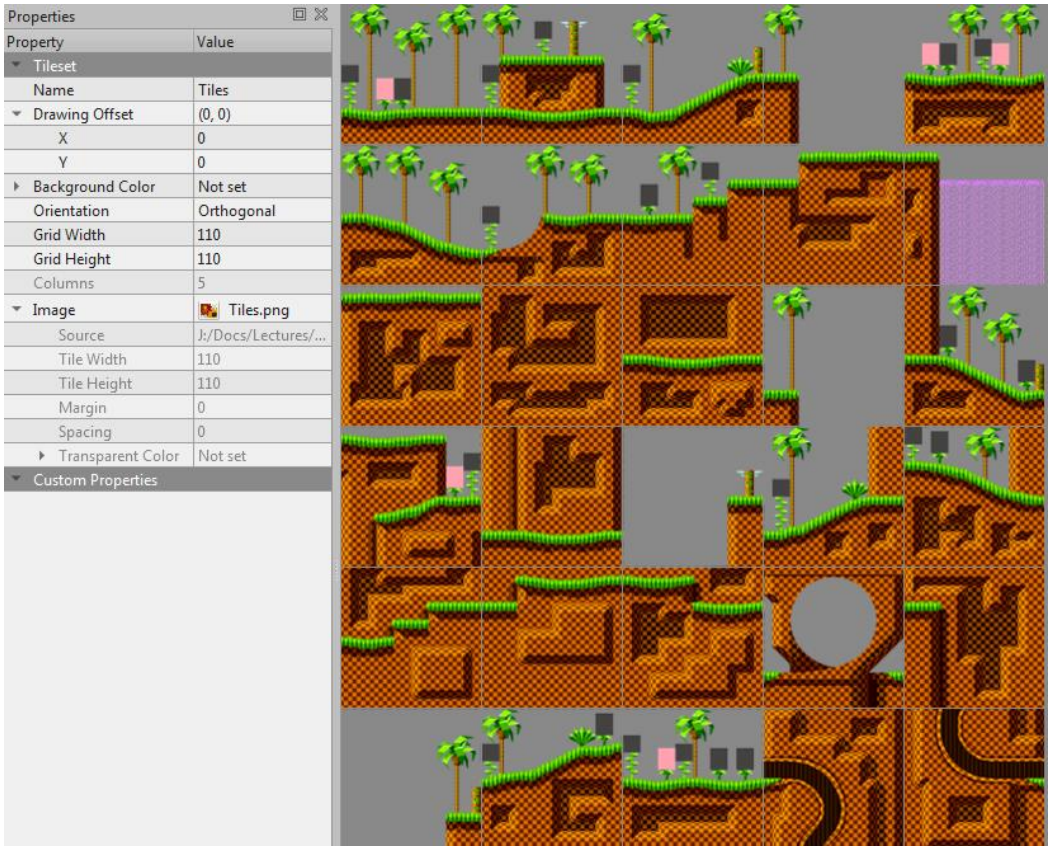
Στο δεύτερο step θα υλοποιήσετε όλες τις λειτουργίες για το grid map και θα κάνετε έναν ενδελεχή έλεγχο όλων των λειτουργιών.

Ειδικά στην περίπτωση του Sonic θα γίνει χρήση των ευκολιών που παρέχει ο Tiled editor ώστε το grid να γίνει edited ως ξεχωριστό map (με ένα layer) και έπειτα να γίνει loaded από csv file. Τα βήματα είναι τα εξής:

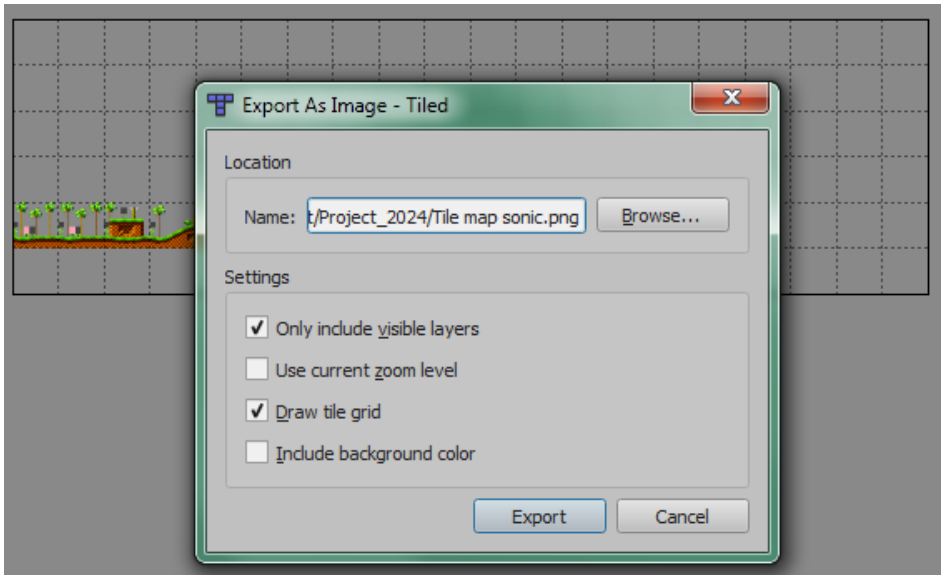
1. Δημιουργήστε το κύριο map και το βασικό foreground layer. Θα φτιάξετε ένα tile set το οποίο θα είναι το Image με τα κύρια μεγάλα tiles toy sonic. Στην παρακάτω εικόνα βλέπετε την επεξεργασία του tile set με το gimp ώστε να περιέχει 5 (W) x 11 (H) big main tiles (οριζόντια x κάθετα) με μέγεθος 110 x 110 pixels το κάθε ένα tile (δηλαδή 550 x 1210 το tile image). Αυτή η εικόνα ως png θα χρησιμοποιηθεί στο παράδειγμα ως το tile set gia το map (terrain) του sonic.
2. ***ΠΡΟΣΟΧΗ*: εσείς θα πρέπει να χρησιμοποιήσετε 128 x 128 tile dimensions (θα τα κάνετε scale με το gimp), ώστε να είναι δύναμη του 2 όπως απαιτείται στη μηχανή**.



Παρακάτω φαίνεται ήδη η ενσωμάτωση του image ως tile set με τα κατάλληλα properties. Εάν θέλετε μπορείτε να διαλέξετε διαφορετικό μέγεθος για tiles (όχι δηλαδή υποχρεωτικά 110 x 110) ωστόσο επειδή στη συνέχεια θα πρέπει να κάνετε subdivision σε grid elements, είτε μέσω των συναρτήσεων *ComputeTileGridBlocks*(), ή εναλλακτικά (και ενδεχομένως ευκολότερα) μέσω του ίδιου editor, θα πρέπει να μπορεί να διαιρείται με τον επιθυμητό αριθμό των grid elements (εδώ έχουμε επιλέξει 10 x 10 pixel size του grid element, δηλαδή 11x11 grid elements per tile).

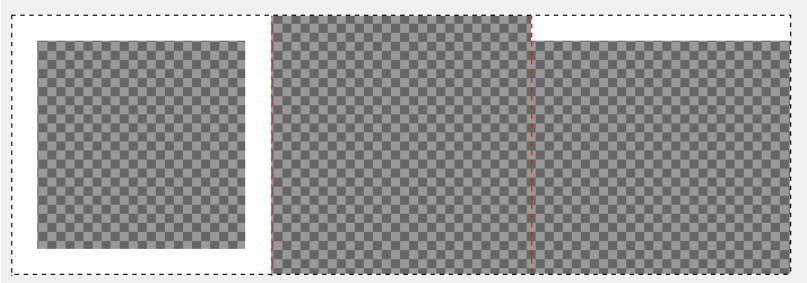


3. Αφού ολοκληρώσετε το map, κάνετε το export ως image, ωστόσο για να διατηρήσει το πλήρες pixel size θα χρησιμοποιήσετε τις παρακάτω ρυθμίσεις. Συνίσταται να έχετε και το grid ώστε να σας οδηγεί κατάλληλα ως προς την ευθυγράμμιση των μεγάλων tiles στο αρχικό terrain.



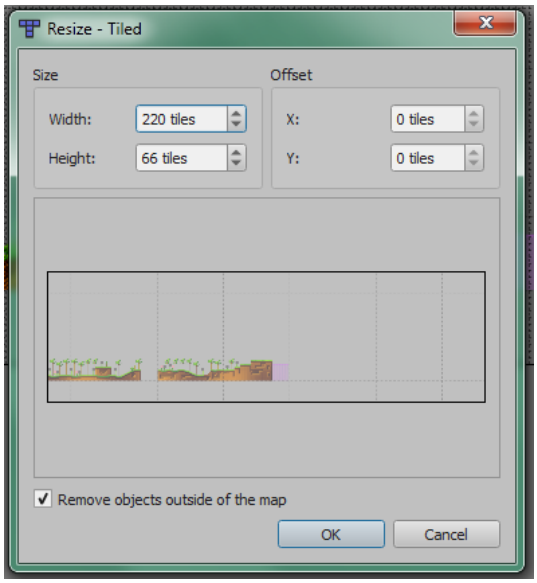
4. Στη συνέχεια δημιουργείτε ένα νέο tile set με μέγεθος tiles αντίστοιχο του subdivision που έχετε επιλέξει για το grid element. Στη δική μας περίπτωση, λόγω των 10 x 10 pixels του grid element, έχουμε φτιάξει ένα πολύ απλό tile image με τρία tiles: solid element (0,0), empty element (0,1), solid top element (0,2), se e;an πολύ μικρό tile image 30 x 10 και σε τοπολογία οριζόντια. Παρακάτω φαίνεται το

tile image όπως δημιουργήθηκε με το gimp – η οπτική μορφή είναι ενδεικτική και έχει σκοπό απλώς να δίνει πληροφορία σε εμάς κατά το editing του layer, εσείς μπορείτε να διαλέξετε διαφορετικό χρώμα (όχι απαραίτητα άσπρο), και ίσως αναπαράσταση. Δε θα χρησιμοποιήσουμε πληροφορία pixels για τον υπολογισμό του grid, αλλά μόνο το index (πχ εδώ είναι 0=solid, 1=empty, 2=solid top).

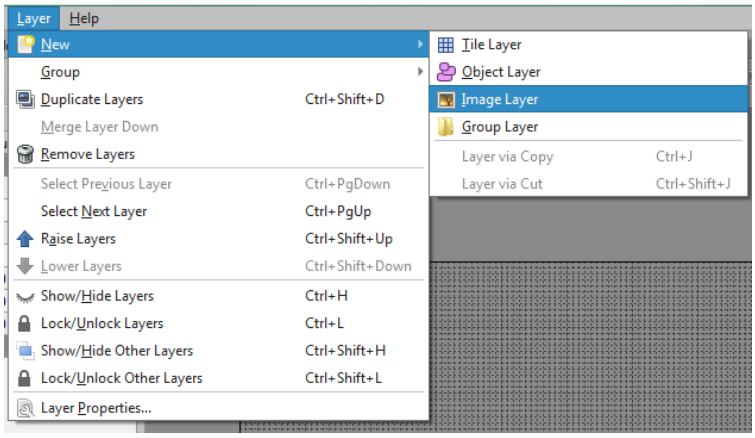


5. Έπειτα φτιάχνετε έναν νέο map με όνομα πχ sonic terrain grid και επίσης εισαγάγετε και το ειδικό απλό tile set για τα grid elements όπως φαίνεται παρακάτω:

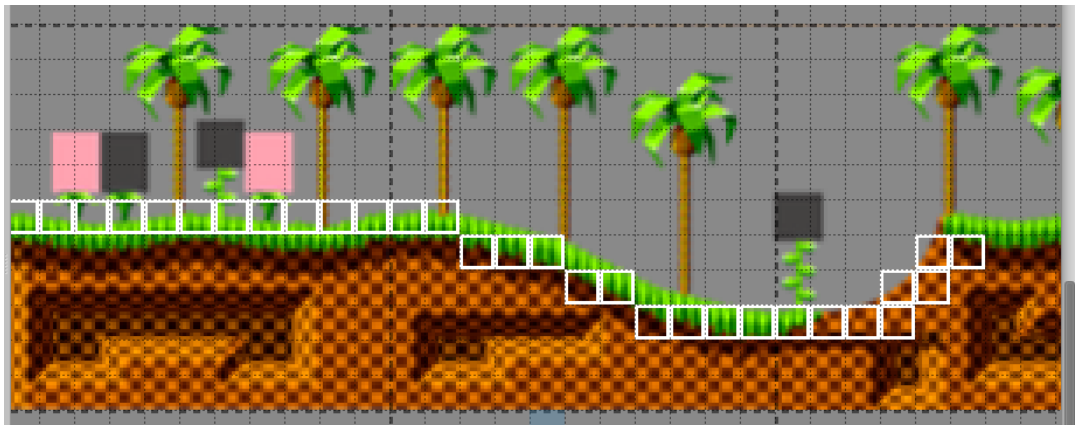
6. Έπειτα δημιουργήστε ένα layer με διαστάσεις αυτές του αρχικού terrain του sonic πολλαπλασιασμένου επί του αριθμού των grid elements σε κάθε διάσταση. Πχ, εάν το terrain έχει 20 (W) x 6 (H) big tiles και έχουμε όπως στο δικό μας παράδειγμα 11 (W) x 11 (H) grid elements per tile, θα ορίσετε ως grid layer διαστάσεις 20 * 11 (W) και 6 x 11 (H), δηλαδή 220 x 66 tiles του grid tile set.

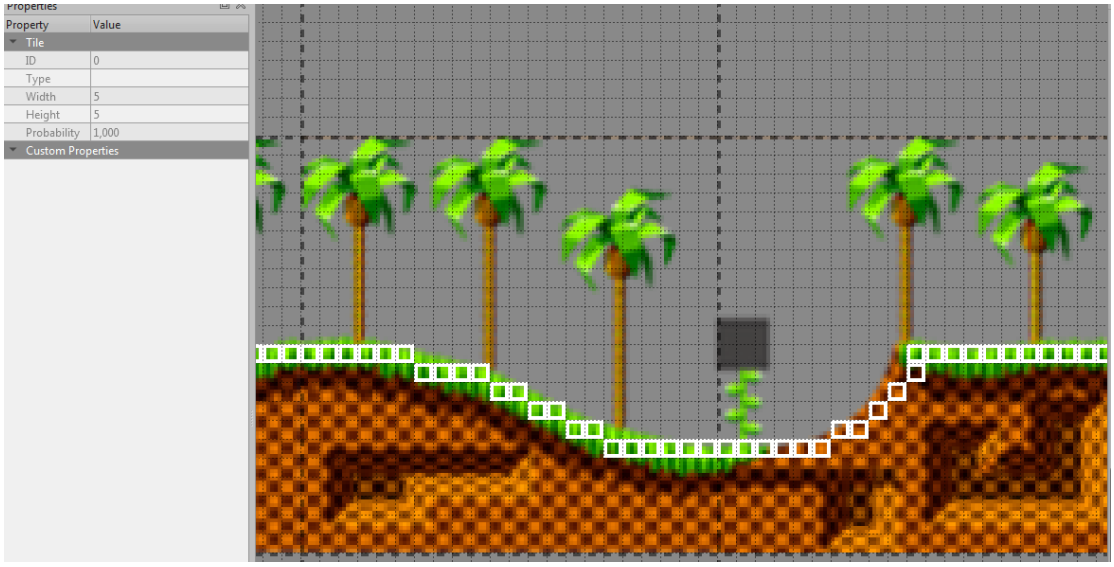


7. Επίσης, δημιουργήστε και ένα image layer στο οποίο θα κάνετε browse και θα επιλέξετε την εικόνα που κάνετε export για το terrain του sonic. Με τον τρόπο αυτό, θα έχετε ως background image (απλώς για editing purposes, βοηθητικό) το terrain ώστε να μπορείτε πλέον να ορίσετε με ακρίβεια και πλήρη ευθυγράμμιση σε αυτό το grid.



8. Στη συνέχεια κάνετε edit το grid με pick και drop tiles από το grid tile set. Θα παρατηρήσετε ότι η ακρίβεια με 10x10 grid element είναι οριακή (τα λευκά boxes είναι του grid layer, το terrain με το γκρίζο grid είναι απλώς το background image που έχουμε θέσει πρωτύτερα). Με 5x5 pixels ως grid element είναι πολύ μεγαλύτερη η ακρίβεια που πετυχαίνουμε, αλλά προφανώς απαιτεί μεγαλύτερο κόπο στο editing. Συγκρίνετε τις δύο παρακάτω εικόνες, η πρώτη με 10x10 grid element ή δεύτερη με 5x5 αντίστοιχα.





9. Εξαγωγή του grid layer σε csv file, και έπειτα custom loading (απλό parsing) του grid από αυτό. Κάθε φορά που αλλάζετε το grid θα πρέπει να κάνετε export και πάλι και loading / parsing αντίστοιχα. Ενδεχομένως λόγω του ότι πρόκειται για πολύ απλής μορφής αρχείο, το loading να γίνεται κάθε φορά, ακόμη και εάν δεν έχει αλλάξει το grid layer (csv) από τη προηγούμενη εκτέλεση. Το αυθεντικό format του Tiled είναι xml, θα είναι κάποιο «.tmx» αρχείο, μέσα στο οποίο θα βρείτε το csv content:

```
<?xml version="1.0" encoding="UTF-8"?>
<map version="1.2" tiledversion="1.3.1" orientation="orthogonal"
renderorder="right-down" compressionlevel="-1" width="440" height="132"
tilewidth="5" tileheight="5" infinite="0" nextlayerid="3" nextobjectid="1">
 <tileset firstgid="1" source="Grid tiles set 5 x 5.tsx"/>
 <imagelayer id="2" name="Image Layer 1">
  <image source="Tile map sonic.png" width="2200" height="660"/>
 </imagelayer>
 <layer id="1" name="Tile Layer 1" width="440" height="132">
 <data encoding="csv">

Εδώ τα comma separated indices, row major, βάσει των width
και height παραπάνω. Το συγκεκριμένο είναι για το grid layer
με 5x5 grid elements και 440 x 132 grid tiles (προφανώς
διπλάσιες διαστάσεις σε σχέση με το 10x10 grid element που
παρουσιάστηκε πριν με 220 x 66 grid tiles)

</data>
 </layer>
</map>
```

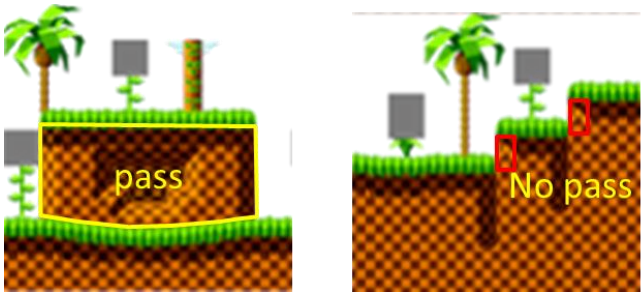
10. Στο παραπάνω συνίσταται να κάνετε copy το cyan highlighted part μόνο, με πρώτα τα width και height και να το κάνετε paste σε ένα txt αρχείο με όνομα **grid_layer_5x5.csv** κάνοντας αντικατάσταση των commas με κενό από το οποίο και θα διαβάσετε τα grid elements εύκολα με ένα double loop (row major), όπως:

```
440 132
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 … συνολικά 440x132 indices
```

Εναλλακτικά μπορείτε να χρησιμοποιήσετε την παρακάτω τακτική για αυτόματο υπολογισμό με subdivision των tiles. Προσοχή, για να έχετε καλό subdivision συνίσταται να κάνετε scale τα μεγάλα tiles του sonic σε 110x110 pixels και να έχετε subdivision με 11x11 grid elements per tile (δηλ 10x10 pixels το κάθε grid element) ή για μεγαλύτερη ακρίβεια με 22x22 grid elements per tile (δηλ 5x5 pixels το κάθε grid element).

Step 2 Alt: Automatic Grid Computation

Για να γίνει αυτό θα πρέπει να επιλύσουμε πρώτα την αμφισημία κάποια τμήματα των tiles να θεωρούνται solid ενώ με την ίδια μορφή σε άλλα σημεία να μη θεωρούνται. Πχ στην παρακάτω εικόνα έχουμε διαβατά τμήματα με την ίδια οπτική στο ένα tile και όχι στο άλλο:



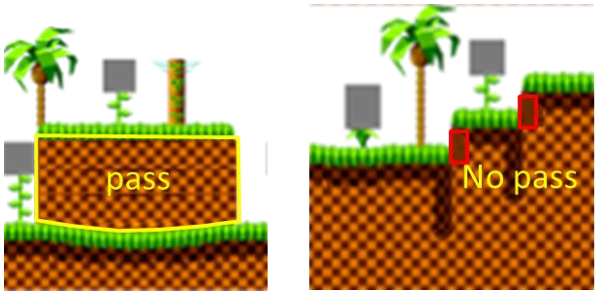
Επίσης, στην περίπτωση των tiles έχουμε δύο επιλογές:

(α) να θεωρήσουμε το γρασίδι (πράσινες αποχρώσεις) ως solid

(β) να θεωρήσουμε το σκούρο χρώμα (βαθιές καφέ αποχρώσεις) ως solid.

Και στις δύο περιπτώσεις θα πρέπει να φτιάξουμε ένα αντίγραφο του tile set bitmap το οποίο θα το επεξεργαστούμε με το gimp και στη συνέχεια αυτό θα δώσουμε ως είσοδο στις συναρτήσεις αυτόματου υπολογισμού του grid από το tile set. Προφανώς δε θα είναι αυτό που θα χρησιμοποιηθεί για το rendering, αλλά μόνο για grid computation.

Άρα, εάν επιλέξουμε πράσινες αποχρώσεις για solid πρέπει να χρωματίσουμε αντίστοιχα όλα τα σημεία. Ομοίως εάν επιλέξουμε το βαθύ καφέ. Στη συνέχεια θα πρέπει στον `TileColorsHolder` `είτε να` να εισάγετε τα χρώματα που θεωρείτε solid ή να φτιάξετε έναν κατάλληλο predicate functor για την `SetInPred` που λαμβάνει το χρωματολόγιο σας υπόψη. Παρακάτω φαίνεται το post-processing των παραπάνω δύο tiles ώστε να προκύπτει το επιθυμητό grid (ανοικτή απόχρωση για empty, dark για solid). Προσοχή, η οπτική ποιότητα δεν μας απασχολεί.



Στη συνέχεια, εφόσον έχετε κάνει όπως αναφέρθηκε πριν τα tiles σε 128x128, διαλέγερτε ένα grid subdivision (4x4 grid elements είναι εντάξει), εφαρμόζεται ο αυτόματος τρόπος υπολογισμού, ειδικότερα, θα κάνετε τα εξής βήματα:

1. ***Να παράγετε το grid απευθείας από τα tiles*** (δοκιμάστε τόσο την *ComputeTileGridBlocks1* όσο και την *ComputeTileGridBlocks2*), επιλέγοντας επίσης έναν σύνολο από indices του tile set τα οποία θα θεωρηθούν ως empty (*να παρέχονται με κάποιο τρόπο στον υπολογισμό όπως θέλετε, αποδεκτό να είναι constant array μέσα στον κώδικα*).
    - Επίσης μπορείτε να εκμεταλλευτείτε και την `SetInPred` της κλάσης `TileColorsHolder` για μεγαλύτερη ευελιξία στον χαρακτηρισμό των χρωμάτων ως empty
    - Καλό είναι στην παραπάνω περίπτωση να χρησιμοποιήσετε το gimp ή κάποιο paint program ώστε νε έχετε μία εικόνα για το φάσμα των RGB color values που θα χαρακτηρίσετε ως empty - πχ το γρασίδι θα έχει μία τάση το G component να είναι πιο έντονο σε σχέση με τα R και B, δηλαδή πρέπει να κάνετε συσχέτιση και όχι απλό τεστ μόνο για το G component
    - Το grid θα πρέπει να αποθηκεύεται είτε ως csv (text) ή σε binary – συνίσταται ως binary και με ένα άμεσο buffer write ώστε να αποφύγετε parsing.
    - Υλοποιήστε την επεξεργασία υπολογισμού υπό συνθήκη at startup, μόνο όταν συντρέχει λόγος υπολογισμού εκ νέου του grid.
    - Σε κάθε περίπτωση, πάντα θα γίνεται loaded στη συνέχεια.

```
…
if (need_to_compute_grid()) {
      compute_grid()
      store_grid();
}
load_grid();
…
```

2. ***Ενσωματώστε τη λειτουργικότητα του προηγούμενου App*** με κατάλληλο τρόπο, αυτό σημαίνει ότι εάν δεν κάνετε τίποτα άλλο, αυτό το version τρέχει ακριβώς σαν να ήταν το προηγούμενο.
3. ***Προσθέστε στο loading function και το loading του grid*** (προγραμματιστικά είτε μέσω derivation από το App subclass ή μέσω composition κάνοντας field το προηγούμενο App subclass - εσείς επιλέγετε).
4. ***Για τον έλεγχο του grid θα υλοποιήσετε ένα movable rectangle***, με διαστάσεις της επιλογής σας, το οποίο θα τοποθετήσετε σε μία αρχική θέση στο terrain που δεν αντιστοιχεί σε solid grid elements, και θα το μετακινείτε interactively με input control (επιλέξτε τα input key bindings, συνίσταται προφανώς να είναι τα cursor keys). Η μετακίνηση θα ελέγχεται μέσω των filter functions (***που πρέπει να συμπληρωθούν και για vertical direction***).
    - ***Προσθέστε στο display function και τη ζωγραφική του rectangle*** (με χρήση rectangle drawing του library, όχι ως bitmap)
    - ***Post rendering***. Δε χρειάζεται να τροποποιήσετε το Game class ώστε να επιτρέπεται κάποιο post render functor, αρκεί η υλοποίηση του render logic να εμπεριέχει και τη ζωγραφική του grid.

5. Δοκιμάστε και ελέγξτε την κίνηση του rectangle σε όλα τα σημεία του terrain. Έτσι θα έχετε μία εικόνα για το πόσο ικανοποιητική είναι η ακρίβεια του grid.
    - ***Να υποστηρίξετε interactive switching μέσω ενός toggle keystroke*** (επιλογής σας), μεταξύ ενός rectangle outline και ενός filled rectangle, ώστε να έχετε καλύτερη εικόνα για τα όρια κίνησης.

Step 3: Implement Layers and Parallax Scrolling

Σε αυτό το step θα πρέπει να γίνει η υλοποίηση των tile layers ως ξεχωριστές κλάσεις καθώς και η υλοποίηση του scrolling. Τα βήματα είναι τα εξής:

1. ***Υλοποίηση των multiple layers***. Σε αυτή τη φάση, για λόγους early testing, μπορείτε να χρησιμοποιήσετε ένα action layer μόνο.
    - Στη συνέχεια ***υλοποιήστε ένα κλασικό action layer***. Για το σκοπό αυτό σας δίδεται στο folder το tile set του Sonic, του 1ου map και κάποια ακόμα που θα χρησιμοποιηθούν σε μετέπειτα φάσεις.
    - Ως προς το **background**:
        i. είτε ένα non-circular bitmap background με μπλε ουρανό και συννεφάκια που να μοιάζει με το αυθεντικό παιχνίδι, ενώ να εφαρμόζετε scrolling ως ένα fraction (configurable) του action layer (δηλαδή perspective proportional scrolling)
        ii. ή ένα horizon layer (επειδή είναι πολύ απλό το background του Sonic και δε θυσιάζονται πολλά tiles – προτεινόμενη λύση, διαστάσεις ίδιες με το action layer, χωρίς perspective)
2. ***Υλοποίηση του terrain με τον tile editor και loading.*** Το terrain του Sonic <u>δεν έχει συνεχόμενη ροή</u>, αλλά αν και κατασκευάζεται ως ένα layer. Αυτό σημαίνει ότι υπάρχουν δύο σκηνές σε ένα layer, τα όρια των οποίων είναι γνωστά και απαγορεύεται κατά τη διάρκεια του gameplay να γίνεται scrolling εκτός των ορίων της παρούσας σκηνής. Οι παρακάτω εικόνες αποτυπώνουν τις ιδιαιτερότητες του terrain.
3. Υλοποίηση της νέας λογικής για scrolling που εξαρτάται μόνο από τη θέση του Sonic στο terrain. Για το λόγο αυτό θα πρέπει να χρησιμοποιήσετε τη θέση του Sonic για τον υπολογισμό της θέσης του view window. Επίσης θα πρέπει να προσθέσετε στους ελέγχους των *filter scroll* methods που εξαρτάται μόνο από τα layer bounds (σε pixel dimensions) με προαιρετικό έλεγχο από ένα scene rectangle το οποίο γίνεται set δυναμικά (<u>precondition: το scene rectangle εμπεριέχεται στο tile layer και είναι πάντα τουλάχιστον όσο το view window</u>).

Figure 1: Picture of the whole first map.

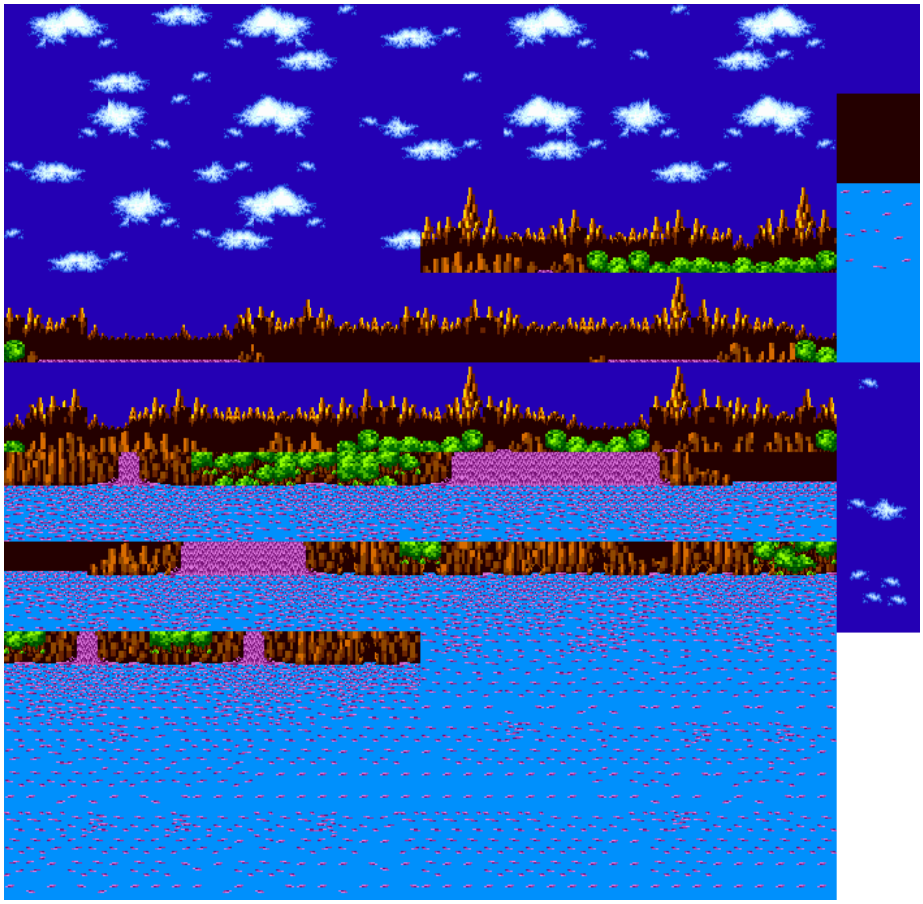


Figure 2: Basic tile set used for the background and foreground of the first map.

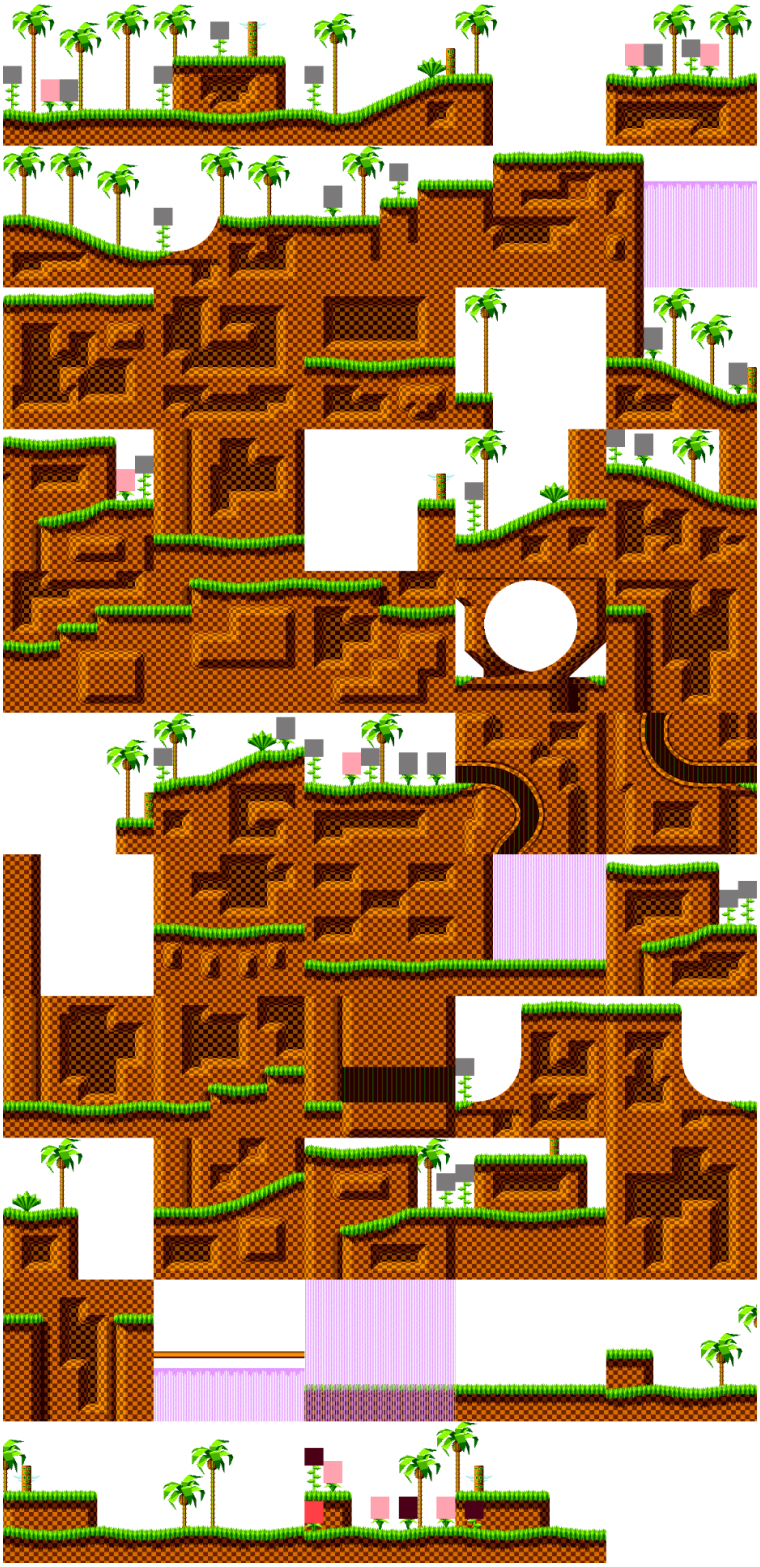


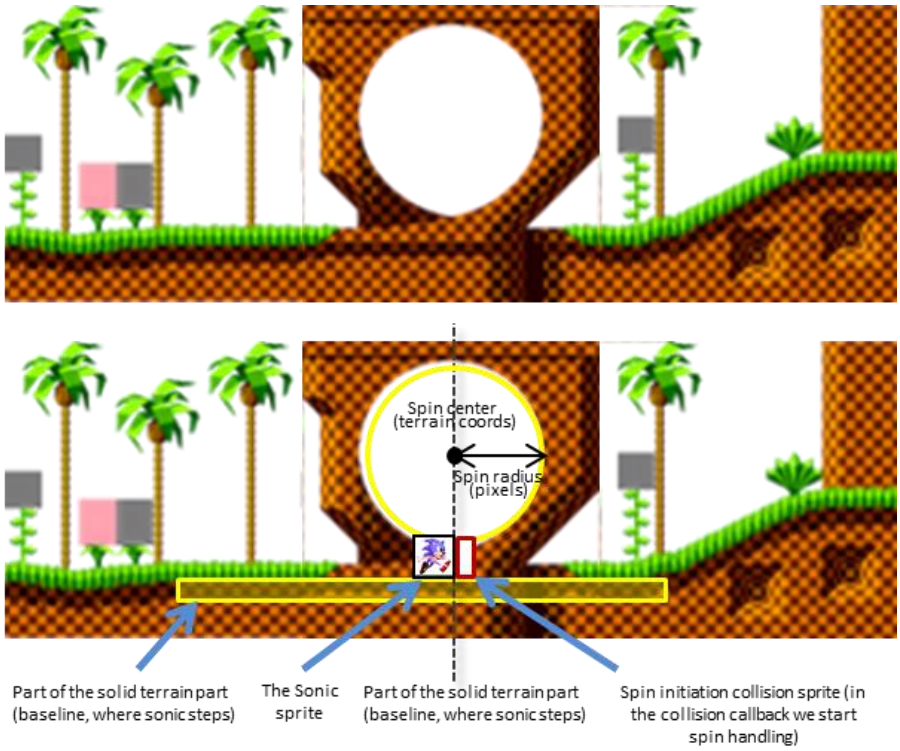
Figure 3: Basic tile set used for the terrain of the first map.

Step 4 : Spin Loops (προαιρετικό, default→bypass loop)

Η διαχείριση των spin loops είναι αρκετά απαιτητική, γίνεται με ειδικό τρόπο, με τη βοήθεια ενός analogue `TickAnimator`, δηλαδή η Progress θα καλείται σε κάθε game loop, και επίσης υλοποιούμε μία ειδική κλάση `SpinLoopHandler` που περιλαμβάνει όλη την απαιτούμενη λειτουργικότητα. Γενικά, η υλοποίηση βασίζεται σε τρία κύρια events:

- **Begin spin loop**: μέσω collision callback με ένα hidden sprite στο σημείο που επιθυμούμε να ξεκινάει το loop (θέλει λίγο ρύθμιση η τοποθέτηση ώστε να γίνεται σωστά η εκκίνηση του loop)
- **End spin loop**: όταν ολοκληρώνεται ένα loop (βάσει της γωνίας, δηλαδή φτάσαμε στις 360 μοίρες ή στης 720 μοίρες σε διπλό Loop) και ή δεν έχουμε activated command για συνέχιση του loop (εάν είμαστε στο πρώτο spin) ή έχουμε ολοκληρώσει και τη δεύτερη περιστροφή
- **Progress spin loop**: μέσω animator που στην action callback τα υπολογίζει όλα και κάνει ότι απαιτείται.

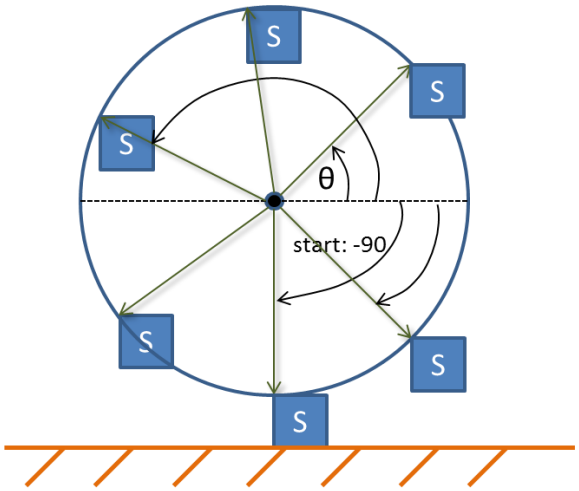
Παρακάτω φαίνεται μία πρώτη άποψη του τρόπου χειρισμού. Η ακτίνα περιστροφής θα θέλει επίσης και αυτή λίγο ρύθμιση ώστε να έχουμε το βέλτιστο αποτέλεσμα. Λάβετε υπόψη ότι ο Sonic τοποθετείται στα σημεία του κύκλου βάσει το top-left origin. Επίσης, στην παρακάτω εικόνα, γίνεται η υπόθεση ότι το γρασίδι δεν αφορά solid grid elements, ενώ δεν έχουμε φτιάξει ascending grid στο σημείο (δηλαδή είναι εντελώς flat). Αυτές είναι απλώς παραδοχές, οι οποίες δεν είναι υποχρεωτικές.

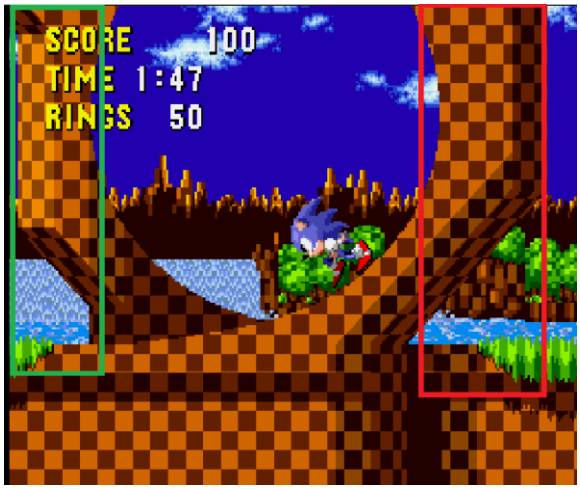


Πχ, στην παρακάτω εικόνα το grid ορίζεται με τρόπο που στην είσοδο του spin loop εξομοιώνει slopes, που σημαίνει ότι και το collision sprite αλλά και η ακτίνα περιστροφής θα πρέπει να προσαρμόζονται στις αναμενόμενες λόγω του grid θέσεις του Sonic.

Η τοποθέτηση του χαρακτήρα γίνεται με τη χρήση της απλής εξίσωσης κύκλου σε raster coordinates ως προς συγκεκριμένο κέντρο, όπως φαίνεται στην παρακάτω εικόνα.

Προσοχή θέλει το γεγονός ότι η εξίσωση υπολογίζει το σημείο του κύκλου που αφορά το origin του Sonic (δείτε στην παρακάτω εικόνα την τοποθέτηση του S box), που σημαίνει ότι θα πρέπει να φροντίσετε ανάλογα με τη γωνία να θέσετε και το κατάλληλο frame του Sonic ώστε να έχει την καλύτερη οπτική απόδοση, αλλά και ενδεχομένως να εφαρμόσετε περιστροφή (ωστόσο επειδή στα αυθεντικά animation films υπάρχουν frames με κατάλληλη κλίση / γωνία ίσως δεν είναι τελικά απαραίτητη η περιστροφή – θα το κρίνετε βλέποντας την ποιότητα της κινησης).





Ο κώδικας της βασικής κλάσης παρέχεται παρακάτω:

```
#define SONIC_GAME
#ifdef SONIC_GAME

using namespace sprites;
using namespace gfx;
using namespace anim;

class SpinLoopHandler final {

private:
	sprites::Sprite*		player		= nullptr;
	sprites::Sprite*		collider		= nullptr;
	anim::TickAnimator*		tickAnimator		= nullptr;
	anim::TickAnimation*		tickAnim		= nullptr;
	Point				centre;
	unsigned			radius		= 0;
	float				rotationSpeedRadians	= 0.0f;
	const double			pi		= 3.14159265359;
	const double			angleRadiansOffset	= -pi * 0.5;	// 90 degrees

	std::function<void(void)>	onBegin;
	std::function<void(void)>	onEnd;
	std::function<void(float)>	onSpin;
	std::function<bool(void)>	predSpinMore;

	float		ToDegrees (float radians) const
			{ return 180.0 * radians / pi; }

	void		Begin (void) {
			tickAnimator->SetOnAction(
				[this](Animator* animator, const Animation& anim) {
					Progress(tickAnimator->GetElapsedTime());
				}
			);
			tickAnimator->Start(*tickAnim, GetGameTime());
		}

	void		End (void)
			{ tickAnimator->Stop(); }

	void		Progress (unsigned totalDelay) {

			auto radians	= ComputeAngleRadians(totalDelay);
			auto pos	= ComputePosition(radians);
			auto degrees	= ToDegrees(radians);
			auto done	= CheckIfSpinsCompleted(&degrees);

			AlignPlayer(pos, degrees);
			if (done)
				End();
		}
```

```
void            CreateCollider (const Rect& collisionBox) {
                        // we make a custom single frame bitmap and respective film
                        auto    bmp     = BitmapCreate(collisionBox.w, collisionBox.h)
                        auto*   film    = new AnimationFilm("film.collider.dynamic");
                        film->SetBitmap(bmp);
                        film->Append(
                                { 0, 0, collisionBox.w, collisionBox.h }
                        );
                        collider = new Sprite(collisionBox.x, collisionBox.y, film);
                        collider->SetVisibility(false);
                        CollisionChecker::GetSingleton().Register(

                                collider,
                                player,
                                [this](Sprite* collider, Sprite* player) {
                                        CollisionChecker::GetSingleton().Cancel(collider, player);
                                        Begin();
                                }
                        );
                }

void            CreateAnimatorAndAnimation (void) {
                        tickAnimator = new TickAnimator();
                        tickAnim = new TickAnimation("tick.anim", 0, 1, false);
                }

const Point     ComputePosition (float angleRadians) const {
                        angleRadians += angleRadiansOffset;  // always offset -90
                        auto x = centre.x + float(radius) * std::cos(angleRadians);
                        auto y = centre.y + float(radius) * std::sin(angleRadians);
                }

float           ComputeAngleRadians (unsigned totalDelay) const
                        { return float(totalDelay) * rotationSpeedRadians; }
void            AlignPlayer (const Point& pos, float degrees) {
                        player->SetPos(pos.x, pos.y);
                        if (onSpin)
                                onSpin(degrees);
                }
bool            CheckIfSpinsCompleted (float* angleDegrees) const {
                        if (*angleDegrees >= 720.0f) {
                                *angleDegrees = 720.0f;
                                return true;
                        }
                        else
                        if (*angleDegrees >= 360.0f) {
                                if (!predSpinMore()) {
                                        *angleDegrees = 360.0f;
                                        return true;
                                }
                                else
                                        return false;
                        }
                        else
                                return false;
                }
```

```
public:
	void		Create (
			const Point&	_centre,
			Dim		_radius,
			const Rect&	_collisionBox,
			unsigned	_fullSpinDelayMsecs,
			Sprite*		_player
		) {
			player = _player;
			radius = _radius;
			centre = _centre;
			rotationSpeedRadians = float(2.0 * pi / double(_fullSpinDelayMsecs));
			CreateCollider(_collisionBox);
			CreateAnimatorAndAnimation();
		}

	void		Destroy (void) {
			BitmapDestroy(collider->GetFilm().GetBitmap());
			delete &const_cast<AnimationFilm&>(collider->GetFilm());
			delete collider;
			delete tickAnimator;
			delete tickAnim;
			collider	= nullptr;
			tickAnimator	= nullptr;
			tickAnim	= nullptr;
		}

	void		SetOnBegin (const std::function<void(void)>& f)
			{ onBegin = f; }
	void		SetOnEnd (const std::function<void(void)>& f)
			{ onEnd = f; }
	void		SetOnSpin (const std::function<void(float degrees)>& f)
			{ onSpin = f; }
	void		SetPredSpinMore (const std::function<bool(void)>& f)
			{ predSpinMore = f; }
};

#endif // SONIC_GAME
```

Action: Power-ups and enemies

Placing Powerups, Items and Bonuses

In the game, coins, spikes, checkpoints and hidden blocks need to interact with the player and perform animations, so they cannot be implemented as mere terrain tiles. You will need to implement them as typical game sprites, so they can be involved in collision checking and interact with the game characters (Sonic and enemies). In order to place them correctly on the terrain, you can do the following trick:

1. Consider the blue background tile that is by default non-solid
2. Construct the terrain normally, using all the tiles you want. In other words, use tiles such as coin, hidden block, spikes or checkpoint tile to mark the precise tile-based positions you want the corresponding game sprites to be actually inserted.
    - This way you see the terrain exactly in the way it is during the game, something that visually happens when game objects are of tile dimensions (as with Sonic)
3. Record somehow the index or coordinates of these tiles in the tile set in order to recognize them when inside the tile map. For example, you will record that the coin tile has the coordinates x:0 and y:9 inside the tile set
4. Just after the terrain is loaded:
    - replace the brick and the other tiles with the background tile – <u>you should provide during grid computation the information that the brick and the other tiles are solid</u>
    - replace the coin tile with background tile – <u>you should provide during grid computation the information that the coin tile is assumed empty</u>
    - record the terrain positions for every such tile instance in order to place a corresponding sprite there - for example, if a coin tile is placed in terrain tile position *(row,col)* you will replace it with the background non-solid tile and create a <u>coin sprite</u> at its pixel-based position
    - you will need set **`SetGridTileBlock`** function here

    This way, your final game terrain will have the same look with the one you actually designed, but will have interactive sprites instead of static tiles where needed. The solid tiles enable characters to walk on them.

    So, when Sonic destroys a hidden block sprite or touches a spike, you should remember to <u>reset the entire underlying grid block corresponding to the tile from solid to non-solid</u>.

    When hidden blocks sprites are hit by Sonic, they make a small bounce animation:

    - During this animation, they can collide with anything that may pass above them (normally only power-ups and enemies)

Placing NPCs

In order to orchestrate the enemies (NPCs) inside the terrain, you are advised to:

1. Have special extra tiles representing the enemies (also record their indices)
2. For some enemies, the sprite frame does not entirely fit within a tile so you will have to do the following:

a. Create an extra tile to represent the enemy and add it to the tile set
b. Place the tile at the bottom-left tile map position of the actual character position (this the tile is aligned bottom-left with respect to the enemy sprite that will be created there)
c. When creating the sprite you should realign its position to fit the bottom and left line of the tile as expected (this is easy). Exception is the piranha that should be actually positioned below designated tile (outside the visible screen) so that initially appears "below" the waters and then emerges-up with an animation.

3. Use the trick from the previous section to replace the NPC-representing tiles with the background ones (not solid) during loading and position the actual NPC sprites at the correct terrain position using the actual tile position (for enemies fitting into one tile).

Supporting Pause / Resume

1. Επεκτείνετε το Game έτσι ώστε να περιλαμβάνει pause/resume:

```
class Game {

        Action        pauseResume;
        bool          isPaused = false;
        uint64_t      pauseTime = 0;

        void   SetOnPauseResume (const Action& f)
               { pauseResume = f;}
        void   Pause (uint64_t t)
               { isPaused = true; pauseTime = t; Invoke(pauseResume); }
        void   Resume (void)
               { isPaused = false; Invoke(pauseResume); pauseTime = 0; }
        bool   IsPaused (void) const
               { return isPaused; }
        uint64_t GetPauseTime (void) const
               { return pauseTime; }
};
```

2. Τροποποιήστε το game loop iteration για να λαμβάνει υπόψιν του το pause / resume:

```
        Render();
        Input();
        if (!IsPaused()) {
```

```
            ProgressAnimations();
            AI();
            Physics();
            CollisionChecking();
            CommitDestructions();
            UserCode();
      }
```

3. Όταν το παιχνίδι είναι σε κατάσταση paused, να εμφανίζεται ένα επιπλέον μήνυμα που να ενημερώνει τον παίκτη κατάλληλα
4. Επεκτείνεται τον AnimatorManager για να περιλαμβάνει την παρακάτω μέθοδο (time shifting):

```
      void  TimeShift (unsigned dt) {
            for (auto* a : running)
                  a->TimeShift(dt);
      }
```

5. Προσθέστε μία εντολή εισόδου (όπως το *ESC* key) το οποίο έχει τον ρόλο της υλοποίησης του pause / resume ως εξής:

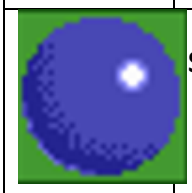
```
// initially
void InstallPauseResumeHandler (Game& game) {
      game.SetOnPauseResume(
            [&game](void) {
                  if (!game.IsPaused()) // just resumed
                        AnimatorManager::GetSingleton().TimeShift(
                              GetGameTime() - game.GetPauseTime()
                        );
            }
      );
}

// in response to the pause / resume input key
void HandleToglePauseResume (Game& game) {
      if (game.IsPaused())
            game.Resume();
      else
            game.Pause(GetGameTime());
}
```

Action: Characters

Sonic

Image	State	Description
	Sonic	Sonic's default state. If he loses to an enemy, he loses all rings. When he collects a super power he becomes "Invincible". Starts walking and gradually builds speed. If he reaches a certain velocity sonic goes to the "full speed" state. If sonic jumps he

	goes for the duration of the jump to “ball” sonic.
Sonic Full Speed	When sonic reaches full speed he swaps animation to that of full speed. He can go through loops now. If sonic jumps he goes for the duration of the jump to “ball” sonic.
 **Sonic ball (Jump)**	Sonic's hitbox is halved and in this state where Sonic can kill enemies. At the end of the jump sonic returns to the previous state he was prior to the jump action.
Sonic invincible	No matter the state, sonic can pick up a power that makes him invincible. In all previous 3 states it's achievable. Invincibility sparkles appear above Sonic's sprite and he cannot be damaged by enemy entities or spikes.

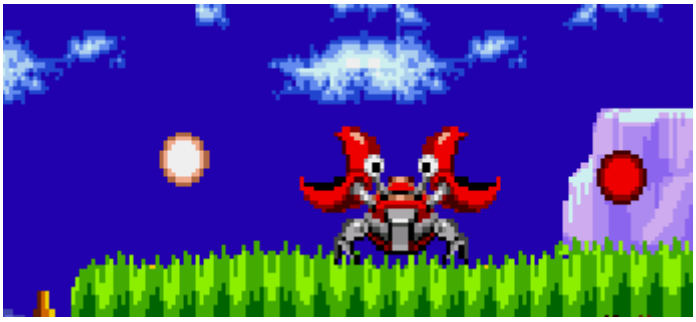
When Sonic loses all rings, he does lose. If he is down on 0 rings and is again hit by an enemy entity, then, if no checkpoints were passed he starts from the start of the level, otherwise he starts at the last checkpoint he passed. The level has an internal **checkpoint**. Once the player has reached (or passed) this checkpoint and Sonic loses with 0 rings, the game restarts from there (you do not need to save the game state when the player crosses the checkpoint. All enemies are back. Upon restarting, you just have to set the view window at the checkpoint instead of the beginning of the level. The checkpoint is a visible structure. When sonic passes a checkpoint, the sprite must play an animation to clarify that it's activated.

Enemies

Image	Name	Points
	Masher / Batabata	100
	Crabmeat / Ganigani	100

	Buzz Bomber / Beeton	100
	Motobug / Motora	100

- ***Masher:*** These "piranhas" go for Sonic in a straight line, and are easily trounced when Sonic jumps on them. They go up and down, their max height is 64 pixels above the bridge while the lowest is just beneath the screen.
- ***Crabmeat:*** They move like crabs left and right till they hit an obstacle (usually a rock) or till they reach a cliff, then they turn the other way around and move. If sonic stands close to them they throw to projectiles, one from each hand that follow a curved path (not targeting Sonic)



- ***Buzz Bomber:*** A flying bee-like creature that will appear when Sonic reaches certain points in the map. They will fly toward sonic and fire a projectile that is targeted at sonic's feet. If they are not killed they fly till they go off screen then they flip and try the same attack pattern. If Sonic skips them by moving too fast and ignores them they don't try to chase him, instead they reset. (If sonic goes 1 tile away without them being killed they reset).

- ***Motora:*** A bug on a motor that goes left and right. Stops for a while and the turns around to continue the loop if it meets an obstacle or a cliff.

All enemies hurt Sonic when the collision occurs sideways. All enemies die if they touch Sonic while he is in Ball state or Invincible. After an enemy dies a small critter is freed from within it.

Power-ups

A lot of power-ups can be found in the Sonic game. They can be hidden behind the top of a palm tree or can be found in the field. They can be broken when Sonic touches them while in ball state. A small animation is played when they are seen switching between static and the reward. When collected another small animation is played where the power goes outside the "screen" and then is awarded to Sonic.

Image	Name	Effect
	Rings	10 rings are awarded to Sonic.
	Invincibility	Sonic becomes invincible for 20 seconds. An animation of stars rotating around Sonic and a change in music indicates that the power is active.
	Speed	Sonic's terminal velocity is increased for 10 seconds allowing him to run even faster than before.
	Shield	Sonic ignores 1 hit and the shield is lost. This buff is active for 30 seconds. Shield is a sprite projected on top of Sonic. The animation is played for 1 frame then disappears for the next and then reappears giving a flickering effect.
	Health Up	Sonic gains increases the remaining health by 1.

The game world is fully constructed by tiles that are called blocks. Rings, spikes, springs etc. are permanent on game terrain and can be implemented via mere tiles. However, some blocks have special behavior and need to interact with Sonic according to collision detection. These are the following:

Interactive Blocks - Entities

Image	Name	Abilities
	Ring	Collectible by Sonic, it acts as a life. If Sonic has 0 rings and takes damage he loses one life the game resets to the most recent checkpoint or

		from the beginning.
	Spikes	Damages Sonic even if sonic is in ball state. The only way to not take damage is if in the invincible state.
	Spring	If sonic gets above he jumps higher than usual. His jump uses his velocity. If Sonic was stationary above it he makes a complete vertical jump while if Sonic had some speed his Jump takes a curved direction.
	Checkpoint	When Sonic passes a checkpoint, a brief animation is played to indicate the activation of the checkpoint. After that if Sonic loses a life he is returned back to the last activated checkpoint. An already active checkpoint cannot be re activated.

Action: Rings, damage and invincibility frames after damage.

When sonic touches an enemy and takes a hit all his rings are scattered and remain in the map (bouncing). After some seconds the rings disappear. When Sonic takes damage he has some invincibility frames that give the player the ability to recover control and try to collect back as many rings as possible.

The rings are scattered in a double circle around sonic and then are given a random velocity. They blink and are bouncing on the floor for some seconds before disappearing. **WARNING** if Sonic carries more than 32 rings, the rest are kept by Sonic. If Sonic has 0 rings then he loses 1 life and returns to the nearest checkpoint or at the start of the map.

When the Player has rings and collides into a Badnik (while uncurled) or other hazard, such as spikes or a field of lava, they are flung backwards and lose their rings. During this special hurt state, their physics behave quite differently. The Player becomes invulnerable for a short time. Speed is set to the opposite of what the player had (e.g if I took damage going forward, in the hurt animation I am moved backwards). The direction their sprite is facing, however, does not change. While in the hurt state, the Player does not react to controller input. Also, Gravity is changed from the normal value to fixed value until they land. When they do land, their X Speed is set to 0, so that they stop completely. The Player cannot leave the hurt state until they land on the ground or on a Solid Object. When the Player leaves the hurt state, Gravity is returned to normal. When the Player has no rings and touches a harmful object, they will enter their death state. X Speed is set to 0, and Y Speed to -7. Gravity is set to the normal value.

Optional Configuration Parameters

You may keep some game attributes into configuration files. Altering configuration values may make your game look completely different. Some of these attributes can be:

- Initial player lives
- Score points each enemy worth
- Number of enemies
- Speed of enemies
- Duration of power ups
- Duration of invincibility in hurt state
- Amount of maximum rings lost on damage taken
- Available time for completing the levels
- Max number of coins kept by Sonic
- Sound effect files and background music theme

Optional Extras

- Extended terrain territory
- Extra platforms
- Damage effects (like animated tinting in red)
- More detailed animation films (more frames and better visuals) for Sonic
- Coin collection effects
- Anything else that adds to the gameplay experience

Ομαδική εργασία -παράδοση

Η εργασία μπορεί να περατωθεί από ομάδα τριών (3) το πολύ ατόμων. Η υλοποίηση της εργασίας είναι υποχρεωτικό να γίνει με τη μέθοδο που διδάσκεται στο μάθημα σε C++, οτιδήποτε άλλο δε θα γίνει δεκτό. ***Το ποσοστό της τελικής βαθμολογίας που καταλαμβάνει η εργασία σας είναι 100%***. ***Η παράδοση και εξέταση της εργασίας θα γίνει σε τρείς φάσεις***:

Η εξέταση των εργασιών θα γίνει με κοινή παρουσίαση μπροστά στις υπόλοιπες ομάδες, με περίπου 15-20 λεπτά διαθέσιμα ανά ομάδα. Αυτό θα γίνει προς το τέλος της εξεταστικής περιόδου και θα ανακοινωθεί μέσα στον Ιανουάριο.

Επίσης κατά την εξέταση θα παραδώσετε όλο το project όπως έχει φτιαχτεί στο IDE.

Επίσης θα προηγηθεί ξεχωριστή εξέταση των ομάδων με έλεγχο του κώδικα όπου θα φανεί η συμμετοχή των μελών και θα βαθμολογηθούν αντίστοιχα. Όσοι αποδειχθεί ότι δε συμμετείχαν θα λάβουν μηδέν και θα υπάρχει σχετικό penalty στο συνολικό βαθμό της ομάδας.

Φροντίστε η αρχική οθόνη του παιχνιδιού να περιέχει: τα ονόματα της ομάδας σας με λατινικούς χαρακτήρες και κεφάλαια γράμματα καθώς και την ένδειξη “University of Crete \n Department of Computer Science \n CS-454. Development of Intelligent Interfaces and Games \n Term Project, Fall Semester 2024”.